

Módulo 06 - Clientes, Sessão Django e CSRF

Objetivo

Entender como o projeto consolida autenticação oficial por Django sessions, preserva carrinho anônimo após login, remove suporte a JWT e separa acesso de cliente e staff.

Commits Estudados

- `d5bc892` - base inicial de clientes.
- `a098e76` - pedido vinculado a cliente e login obrigatório no fechamento.
- `a0e6c5b` - cadastro e autenticação de clientes por sessão.
- `eeb170a` - remove suporte a JWT.
- `b4ba3c7` - protege painel admin por sessão staff.
- `059c913` - permite login de staff sem cliente.
- `a9be2e2` - corrige logout de sessões Django.

Aula 1 - Cliente Como Perfil do Usuário

O commit `d5bc892` cria o app `clientes`.

Código para ler:

- `backend/clientes/models.py`
- `backend/clientes/admin.py`
- `backend/clientes/tests.py`
- `backend/setup/settings.py`

Modelo central:

```
class Cliente(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL, related_name="cliente")
    nome = models.CharField(max_length=200)
    telefone = models.CharField(max_length=30)
    ativo = models.BooleanField(default=True)
```

Decisão real:

- O projeto usa o `User` do Django para autenticação.
- Dados de cliente ficam em `Cliente`.
- A relação é 1 para 1.

Testes para estudar:

- `test_cria_cliente_vinculado_a_user`
- `test_impede_dois_clientes_para_o_mesmo_user`
- `test_cliente_admin_registrado_com_configuracao_basica`

Aula 2 - Checkout Exige Login

O commit `a098e76` vincula pedido ao cliente e exige login para converter carrinho.

Código para ler:

- `backend/pedidos/models.py`
- `backend/pedidos/services.py`
- `backend/pedidos/views.py`
- `backend/pedidos/tests.py`

Mudanças reais:

- `Pedido` passa a ter `usuario` e `cliente`.
- `ConverterCarrinhoPedidoView` exige `IsAuthenticated`.
- `PedidoService.converter_carrinho` valida o dono do carrinho.
- Listagem de pedidos filtra por `usuario=request.user`.

No estado atual:

```
class ConverterCarrinhoPedidoView(CarrinhoMixin, APIView):  
    permission_classes = [IsAuthenticated]
```

Testes para estudar:

- `test_anonimo_nao_consegue_converter_carrinho_em_pedido`
- `test_usuario_autenticado_usa_o proprio_carrinho`
- `test_usuario_autenticado_lista_apenas_os_proprios_pedidos`

- `test_usuario_autenticado_nao_acessa_contrato_de_pedido_de_outro_usuario`

Regra crítica:

- Cliente só acessa os próprios dados.

Aula 3 - Cadastro e Login Por Sessão

O commit `a0e6c5b` implementa cadastro, login, logout, `/me/` e CSRF.

Código para ler:

- `backend/clientes/serializers.py`
- `backend/clientes/views.py`
- `backend/clientes/urls.py`
- `backend/clientes/tests.py`
- `backend/setup/settings.py`

Endpoints oficiais:

- `POST /api/auth/cadastro/`
- `POST /api/auth/login/`
- `POST /api/auth/logout/`
- `GET /api/auth/me/`
- `GET /api/auth/csrf/`

Pontos reais:

- `CadastroClienteSerializer` normaliza e-mail.
- Senha passa por validadores do Django.
- `login(request, user)` cria a sessão.
- `csrf_protect` protege cadastro, login e logout.
- `ensure_csrf_cookie` expõe o token para o frontend.
- carrinho anônimo da sessão anterior é vinculado após login.

Trecho de comportamento:

```
login(request, user)
vincular_carrinho_anonimo_da_sessao(request, user, session_key_anterior)
```

Testes para estudar:

- `test_cadastro_cria_user_e_cliente`
- `test_cadastro_valida_senha_fraca`
- `test_login_valido_autentica`
- `test_csrf_retorna_token_e_cookie`
- `test_carrinho_anonimo_e_preservado_e_vinculado_apos_login`

Aula 4 - Remoção de JWT

O commit `eeb170a` remove suporte a JWT e consolida sessão como autenticação oficial.

Código para ler:

- `backend/setup/settings.py`
- `backend/setup/urls.py`
- `backend/clientes/tests.py`
- `AGENTS.md`
- `docs/PROJECT_CONTEXT.md`
- `docs/CODEX_RULES.md`
- `docs/SECURITY_CHECKLIST.md`

Mudanças reais:

- pacote JWT sai de `requirements.txt`;
- endpoint legado de token é removido;
- `REST_FRAMEWORK` fica com `SessionAuthentication`;
- docs passam a proibir JWT sem solicitação explícita.

No estado atual:

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework.authentication.SessionAuthentication",
    ),
}
```

Teste para estudar:

- `test_endpoint_token_jwt_legado_nao_existe`

Lição:

- Remover alternativa insegura ou fora do padrão é uma decisão de arquitetura.
- Autenticação por sessão exige CSRF em métodos mutáveis.

Aula 5 - Staff Sem Cliente e Admin Protegido

O commit `b4ba3c7` protege o painel admin por sessão staff.

O commit `059c913` permite login de staff sem `Cliente`.

Código para ler:

- `backend/clientes/serializers.py`
- `backend/clientes/views.py`
- `backend/clientes/tests.py`
- `frontend/src/components/admin/AdminLayout/index.tsx`
- `frontend/src/services/auth.ts`
- `frontend/src/types/auth.ts`

Ponto real:

- Cliente comum precisa ter perfil `Cliente`.
- Staff pode autenticar sem perfil de cliente.
- `/api/admin/me/` usa `IsAdminUser`.
- Frontend usa esse endpoint para proteger o painel.

No serializer atual:

```
if (
    not authenticated_user.is_staff
    and not authenticated_user.is_superuser
    and not hasattr(authenticated_user, "cliente")
):
    self.fail("invalid_credentials")
```

Testes para estudar:

- `test_login_staff_sem_cliente_autentica_por_email`
- `test_login_usuario_comum_sem_cliente_retorna_erro_generico`
- `test_admin_me_anonimo_negado_com_401`

- `test_admin_me_cliente_comum_negado_com_403`
- `test_admin_me_staff_permitido`

Aula 6 - Logout Correto em Sessões Django

O commit `a9be2e2` corrige logout de sessões Django.

Código para ler:

- `backend/clientes/tests.py`
- `frontend/src/contexts/AuthContext.tsx`
- `frontend/src/components/admin/AdminLayout/index.tsx`
- `frontend/src/components/client/Header/index.tsx`

Problema real tratado:

- Logout precisa encerrar autenticação no servidor.
- Frontend precisa limpar estado local mesmo quando a sessão já expirou.
- Admin precisa perder acesso ao painel depois do logout.

Testes para estudar:

- `test_logout_encerra_autenticacao`
- `test_logout_staff_sem_cliente_encerra_auth_e_acesso_admin`

No frontend atual, o contexto diferencia:

- erro de usuário já deslogado;
- erro de CSRF;
- erro real do servidor.

Revisão do Módulo

Você deve conseguir responder:

- Por que o projeto usa sessão Django e não JWT?
- Onde CSRF é exigido?
- Como carrinho anônimo é preservado depois do login?
- Por que staff pode não ter `Cliente` ?
- Como o backend impede cliente de ler pedido de outro cliente?

Exercício Prático

Abra `git show a0e6c5b` e siga o fluxo de login:

1. serializer autentica;
2. view chama `login`;
3. sessão é criada;
4. carrinho anônimo é vinculado;
5. `/me/` confirma o usuário autenticado.

Depois abra `git show eeb170a` e identifique tudo que foi removido para abandonar JWT.